

ECE/CS 552 Final Project Report – Spring 2025

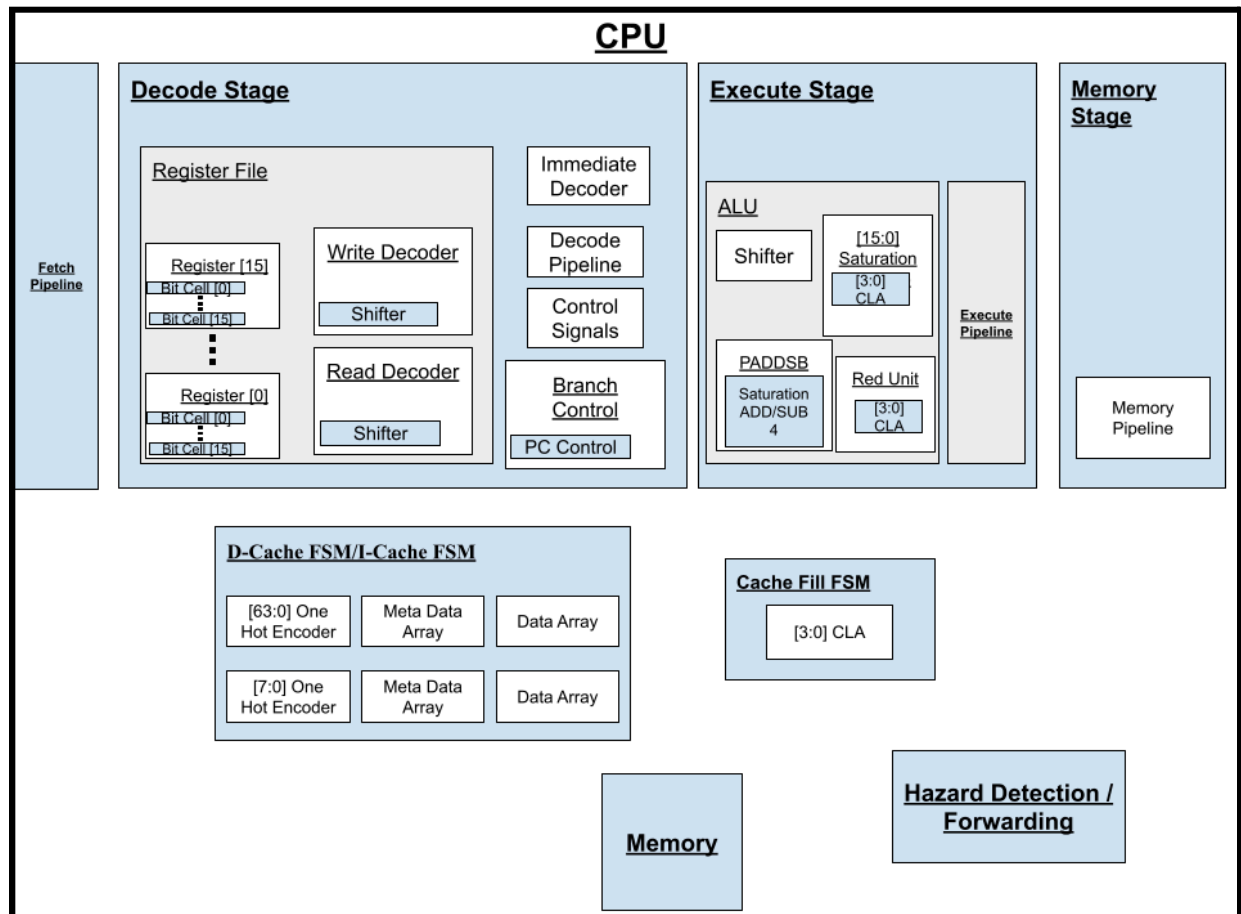
Avinash Anand, Clayton Culbert, Kevin Le, and David Tisch

Section 1: Overview

This project implements a three-phase processor design for the WISC-S25 ISA, culminating in a pipelined architecture with a memory hierarchy.

- Phase 1: Designed a single-cycle processor that executes each instruction in one clock cycle using separate instruction and data memories.
- Phase 2: Extended the design into a 5-stage pipeline (IF, ID, EX, MEM, WB), implementing forwarding, hazard detection, stalling and flushing to manage data and control hazards.
- Phase 3: Integrated a cache hierarchy consisting of a 2-way set-associative I-cache and D-cache, along with controllers and a pipelined memory interface with arbitration logic to handle concurrent memory access requests.

Top-Level Block Diagram:



Section 2: Responsibility/Task Breakdown

Phase 1 – Single-Cycle Processor

During Phase 1, the team collaborated to implement a fully functional single-cycle processor that supports the WISC-S25 ISA. The work was divided evenly among members, with each person taking ownership of key modules and actively contributing to testing, debugging, and overall system integration.

Avinash	Clayton	Kevin	David
Control Signals	ALU Unit	Branch Control	Adder

Phase 2 – 5-Stage Pipelined Processor

For Phase 2, the team transitioned from a single-cycle design to a fully pipelined 5-stage processor architecture. Work responsibilities were again evenly distributed, with each member focusing on a specific pipeline segment or logic component.

Avinash	Clayton	Kevin	David
Execute and Memory Pipeline Units	Fetch and Decode Pipeline Units	Memory and WB Pipeline Units	Hazard Detection and Forwarding

Phase 3 – Cache Hierarchy and Memory Controller

In Phase 3, the team integrated a memory hierarchy into the pipelined processor by implementing 2-way set-associative instruction and data caches, along with corresponding cache controllers and a shared memory interface. Team members worked collaboratively across all components, while each took lead on specific implementation or debugging responsibilities.

Avinash	Clayton	Kevin	David
I-Cache and D-Cache	Cache Controller and FSM Logic (Lead)	Debugging and Final Report	Cache Control & FSM Logic Support, Final Report Support

All members contributed to debugging, waveform analysis, and verification using the provided testbenches to ensure functional correctness.

Section 3: Special Features / Optimization Highlights

Phase 3 Optimization – Parallel Set Access for Cache Tag Checking

To improve cache access efficiency, we modified the MetaDataArray and DataArray modules to each have 64 entries instead of the default 128. This change allows both ways of a set in the 2-way set-associative cache to be accessed in parallel, rather than checking them sequentially across multiple cycles. As a result, we are able to determine cache hits or misses and retrieve the correct data within a single cycle, significantly reducing cache access latency and avoiding unnecessary stalls in the pipeline.

Section 4: Completeness

Our final design meets all the specified requirements for Phases 1, 2, and 3. The processor passed all provided test programs, producing correct outputs and demonstrating expected behavior throughout. There are no missing features or non-functional components in the final implementation. Additionally, the design performed as intended during the individual submission of each phase, and no modifications were necessary between the demo and the final report submission.

Section 5: Testing Methodology

To ensure correctness and robustness at every stage of development, we employed a combination of unit testing, integration testing, and waveform analysis.

Phase 1 – Single-Cycle Processor Testing: We began with unit testing of all fundamental modules, including the ALU, register file, and control logic. Each module was verified independently to ensure correct functionality for a wide range of input cases, including edge cases like overflow and zero results. Once the modules passed their individual tests, we integrated them into a single-cycle processor and used provided test programs to verify end-to-end instruction execution. Outputs were compared against expected register and memory states to confirm correctness.

Phase 2 – Pipelined Processor Testing: For the pipelined version of the processor, testing focused heavily on control and data hazards. We constructed and ran test cases that triggered various hazard scenarios, including RAW (read-after-write) dependencies, control hazards from branches, and conditions requiring pipeline stalls or forwarding. To verify the correct behavior of the hazard detection and forwarding units, we used waveform analysis tools (Modelsim/Icarus) to inspect pipeline register values and data paths cycle by cycle. We also ensured that the HLT

instruction correctly asserted its signal only in the writeback stage and that the PC was properly updated (or held) in edge cases like mispredicted branches.

Phase 3 – Cache Hierarchy and Memory Controller Testing: Testing for this phase focused on cache behavior, miss handling, and memory arbitration. We designed test cases that triggered both I-cache and D-cache misses to validate the functionality of the cache miss FSM and the parallel block fill mechanism. Our modified cache structure enabled single-cycle hit detection and data retrieval, which we confirmed through waveform analysis. We also evaluated simultaneous cache access scenarios to ensure that arbitration logic correctly prioritized requests without causing deadlock. All test programs passed and exhibited the expected behavior.

Section 6: Results

All of our test results passed, a breakdown of each test with its simulation cycle counts is provided below.

Phase/Test	Correct	Cycle Count
Phase 1 Test 1:	Correct	11
Phase 1 Test 2:	Correct	12
Phase 1 Test 3:	Correct	19
Phase 2 Test 1:	Correct	18
Phase 2 Test 2:	Correct	20
Phase 2 Test 3:	Correct	28
Phase 3 Test 1:	Correct	55
Phase 3 Test 2:	Correct	72
Phase 3 Test 3:	Correct	75
Phase 3 Test 4:	Correct	352